

# Chapter 2

## Factor Models and Dimensionality Reduction

Giovanni Della Lunga  
giovanni.dellalunga@unibo.it

Advanced Topics in Artificial Intelligence

Bologna - March, 2026



# Part 1

## Linear Factor Models and PCA

# Part 1 – Roadmap

- 1 The Arbitrage Pricing Theory (Ross, 1976)
- 2 The static linear factor model: setup and interpretation
- 3 Observable factors: Fama–French models
- 4 Latent factors: PCA via Singular Value Decomposition
- 5 Consistency of PCA estimators (Bai & Ng, 2002)
- 6 Choosing the number of factors  $K$
- 7 Observable vs. latent, static vs. conditional: a taxonomy
- 8 Limitations of static models – motivating what comes next

## Subsection 1

### APT and the factor model

# Starting Point: Why Factor Models?

## Fundamental question in asset pricing:

Why do different assets earn different average returns?

## Two possible answers:

- 1 **Risk compensation:** Higher average returns reflect exposure to systematic risks that investors want to be compensated for bearing.
- 2 **Anomalies / Mispricing:** Some assets are “cheap” for reasons unrelated to risk (behavioral biases, institutional frictions, ...).

Factor models formalize the **risk compensation** view: all differences in expected returns arise from differences in exposure to a small number of common risk factors.

## Central tension in Asset Pricing

Do asset characteristics predict returns because they proxy for risk exposures (factor structure) or because they capture mispricing (anomalies)?

# The Arbitrage Pricing Theory (Ross, 1976)

**APT assumption:** Asset returns are generated by a *linear factor structure*.

If there are  $K$  systematic factors, then for asset  $i$  at time  $t$ :

$$r_{i,t} = \alpha_i + \beta_{i,1} f_{1,t} + \beta_{i,2} f_{2,t} + \cdots + \beta_{i,K} f_{K,t} + u_{i,t}$$

**Key ingredients:**

- $r_{i,t}$ : excess return of asset  $i$  (over the risk-free rate)
- $f_{k,t}$ : return on factor  $k$  at time  $t$  (systematic shock)
- $\beta_{i,k}$ : **loading** of asset  $i$  on factor  $k$  (risk exposure)
- $u_{i,t}$ : idiosyncratic return, uncorrelated with factors
- $\alpha_i$ : intercept (“alpha” – pricing error)

**APT conclusion:** If no-arbitrage holds and  $N$  is large, then  $\alpha_i \approx 0$  for all  $i$ . All expected returns are explained by factor exposures.

# The Static Linear Factor Model – Vector Notation

Stacking across all  $N$  assets at time  $t$ :

$$r_t = \beta f_t + u_t$$

## Dimensions:

|         |                               |                                        |
|---------|-------------------------------|----------------------------------------|
| $r_t$   | $\in \mathbb{R}^{N \times 1}$ | vector of excess returns of $N$ assets |
| $f_t$   | $\in \mathbb{R}^{K \times 1}$ | vector of $K$ factor returns           |
| $\beta$ | $\in \mathbb{R}^{N \times K}$ | matrix of factor loadings              |
| $u_t$   | $\in \mathbb{R}^{N \times 1}$ | idiosyncratic errors                   |

## Assumptions:

- $\mathbb{E}[u_t | f_t] = 0$  (idiosyncratic risk is orthogonal to factors)
- $\text{Cov}(u_{i,t}, u_{j,t}) \approx 0$  for  $i \neq j$  (approximate factor structure)
- $\beta$  is **constant over time** (static loadings)



# Interpretation of Each Component

## Factors $f_t$ :

- Sources of *systematic* (non-diversifiable) risk
- Examples: market return, recession shocks, interest rate changes
- Can be **observable** (traded portfolios) or **latent** (unknown)

## Loadings $\beta_i$ :

- How much asset  $i$  “co-moves” with each factor
- $\beta_{i,k}$  high  $\Rightarrow$  asset  $i$  is very sensitive to factor  $k$
- Determines risk *and* expected return

## Idiosyncratic error $u_{i,t}$ :

- Firm-specific shock (CEO scandal, product recall, ...)
- **Diversifiable:** in a portfolio of many stocks,  $\frac{1}{N} \sum_i u_{i,t} \rightarrow 0$

# The Pricing Implication: Security Market Line

Under no-arbitrage (APT), all expected excess returns are compensation for factor risk:

## Asset Pricing Restriction

$$\mathbb{E}[r_{i,t}] = \beta_i' \lambda$$

where  $\lambda = \mathbb{E}[f_t]$  is the  $K \times 1$  vector of **factor risk premia**.

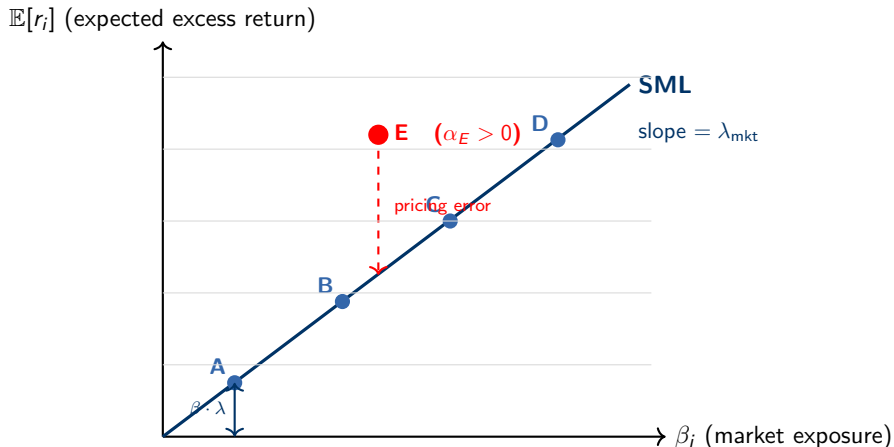
### What this says:

- An asset's expected return depends **only** on its factor exposures  $\beta_i$
- Each unit of exposure to factor  $k$  is compensated at rate  $\lambda_k$
- There is **no alpha**:  $\alpha_i = \mathbb{E}[r_{i,t}] - \beta_i' \lambda = 0$

If  $\alpha_i \neq 0$  for some asset  $i$ , either:

- ① The model is misspecified (wrong factors or wrong  $\beta$ ), or
- ② There is genuine mispricing (anomaly)

# Graphical Intuition: One Factor (CAPM)



Assets on the Security Market Line:  $\alpha = 0$  (correctly priced).

Asset E is above the line: it earns more than justified by its  $\beta \Rightarrow \alpha_E > 0$ .

# Multi-Factor Generalization

With  $K > 1$  factors, the SML becomes a *hyperplane* in  $(K + 1)$ -dimensional space:

$$\mathbb{E}[r_i] = \beta_{i,1}\lambda_1 + \beta_{i,2}\lambda_2 + \cdots + \beta_{i,K}\lambda_K$$

**Example with  $K = 3$ :**

$$\mathbb{E}[r_i] = \underbrace{\beta_{i,\text{mkt}}}_{\text{market exposure}} \cdot \underbrace{\lambda_{\text{mkt}}}_{\approx 6\%/yr} + \underbrace{\beta_{i,\text{smb}}}_{\text{size exposure}} \cdot \underbrace{\lambda_{\text{smb}}}_{\approx 2\%/yr} + \underbrace{\beta_{i,\text{hml}}}_{\text{value exposure}} \cdot \underbrace{\lambda_{\text{hml}}}_{\approx 3\%/yr}$$

**Key question:** What are the “right” factors?

## Approach 1: Observable factors

Specify factors from economic theory or empirical regularities (Fama–French).

## Approach 2: Latent factors

Extract factors statistically from the data (PCA).

## Subsection 2

### Observable factor models (Fama-French)

# Observable Factor Models: The Fama–French Tradition

**Idea:** Construct factors as returns on *long-short portfolios* sorted by firm characteristics.

| Factor | Acronym | Construction                                                                    |
|--------|---------|---------------------------------------------------------------------------------|
| Market | MKT     | Value-weighted market return minus risk-free rate                               |
| Size   | SMB     | Small minus Big: return on small-cap portfolio minus large-cap portfolio        |
| Value  | HML     | High minus Low: return on high book-to-market portfolio minus low B/M portfolio |

These three factors define the **Fama–French three-factor model** (FF3, 1993).

# Extending FF3: Five and Six Factor Models

| Model      | Factors                      | What's added                                                                      |
|------------|------------------------------|-----------------------------------------------------------------------------------|
| CAPM       | MKT                          | Single factor                                                                     |
| FF3        | MKT, SMB, HML                | Size + Value                                                                      |
| Carhart 4F | MKT, SMB, HML, UMD           | + Momentum (Up Minus Down)                                                        |
| FF5        | MKT, SMB, HML, RMW, CMA      | + Profitability (Robust Minus Weak), + Investment (Conservative Minus Aggressive) |
| FF6        | MKT, SMB, HML, RMW, CMA, UMD | + Momentum                                                                        |

# How Are FF Betas Estimated?

Given observable factors  $f_t = (f_{1,t}, \dots, f_{K,t})'$ , for each asset  $i$  run a **time-series regression**:

$$r_{i,t} = \alpha_i + \beta_{i,1}f_{1,t} + \dots + \beta_{i,K}f_{K,t} + u_{i,t}, \quad t = 1, \dots, T$$

**Estimation:** OLS gives  $\hat{\beta}_i = (\hat{\beta}_{i,1}, \dots, \hat{\beta}_{i,K})'$  for each stock  $i$ .

## Practical issues:

- Need enough time-series data per stock (typically 60 months rolling window)
- $\hat{\beta}_i$  is **constant** within each estimation window
- Must re-estimate periodically  $\Rightarrow$  crude step-function approximation to time variation
- Poor for short-lived stocks (IPOs, delistings) – unbalanced panel problem



# Strengths and Weaknesses of Observable Factor Models

## Strengths

- Economically interpretable factors
- Simple estimation (OLS)
- Long history of empirical validation
- Factors are tradeable portfolios  $\Rightarrow$  direct economic meaning
- Widely used in industry and academia

## Weaknesses

- **Which factors?** The “factor zoo” problem – hundreds of proposed factors
- **Static betas:** Cannot capture time-variation in risk exposures
- **No conditioning information:** Ignores firm characteristics
- **Ad hoc factor selection:** No guarantee these are the “right” factors

## Subsection 3

### Latent factors and PCA

# Latent Factors: The PCA Approach

**Alternative:** Don't pre-specify factors. Let the *data* tell us what the common sources of variation are.

**Idea:** Find the  $K$  directions in return space that explain the most *common variance* across all  $N$  assets.

**Setup:** Stack the factor model over all  $T$  periods:

$$R = \beta F + U$$

$R \in \mathbb{R}^{N \times T}$  matrix of all excess returns

$\beta \in \mathbb{R}^{N \times K}$  loadings matrix

$F \in \mathbb{R}^{K \times T}$  factors matrix

$U \in \mathbb{R}^{N \times T}$  idiosyncratic errors

**Problem:** Both  $\beta$  and  $F$  are unknown. We need to estimate them *simultaneously* from  $R$  alone. This is an **unsupervised** problem.

# PCA via Singular Value Decomposition (SVD)

**Step 1:** Demean the returns:  $\bar{R} = R - \bar{r} \iota'$  where  $\bar{r} = \frac{1}{T} \sum_t r_t$ .

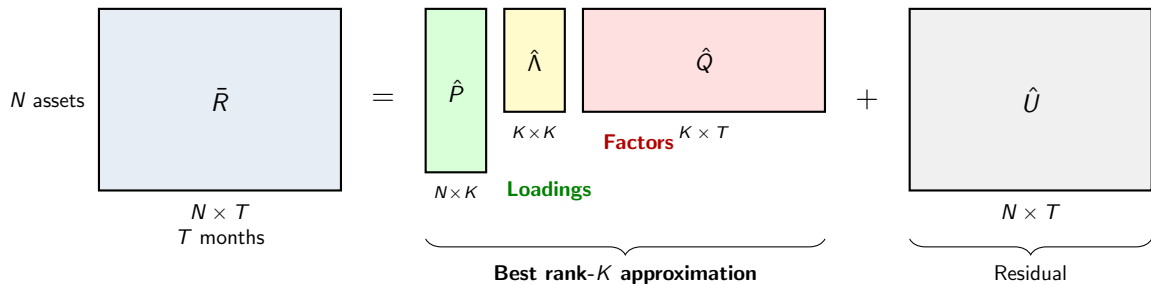
**Step 2:** Compute the SVD of  $\bar{R}$ :

$$\bar{R} = \hat{P} \hat{\Lambda} \hat{Q} + \hat{U}$$

where:

- $\hat{P} \in \mathbb{R}^{N \times K}$ : left singular vectors = **estimated factor loadings**
- $\hat{\Lambda} \in \mathbb{R}^{K \times K}$ : diagonal matrix of singular values ( $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_K > 0$ )
- $\hat{Q} \in \mathbb{R}^{K \times T}$ : right singular vectors = **estimated factors**
- $\hat{U} \in \mathbb{R}^{N \times T}$ : residual matrix

# SVD: Visual Representation



PCA selects the  $K$  directions that capture the **maximum** amount of common variation in the return panel.

# Equivalent Formulation via Covariance Matrix

## Alternative (equivalent) approach:

**Step 1:** Estimate the covariance matrix of returns:

$$\hat{\Sigma} = \frac{1}{T} \bar{R} \bar{R}' \quad (N \times N)$$

**Step 2:** Eigenvalue decomposition:

$$\hat{\Sigma} = \hat{P} \hat{D} \hat{P}' + \text{residual}$$

where  $\hat{D} = \hat{\Lambda}^2 / T$  contains the eigenvalues.

## Connection:

- Eigenvectors of  $\hat{\Sigma}$  = left singular vectors of  $\bar{R}$  = PCA loadings  $\hat{P}$
- Eigenvalues of  $\hat{\Sigma}$  = squared singular values /  $T$  = variance explained by each PC

## No-arbitrage variant

To impose the zero-intercept restriction (no alpha), apply PCA to the *uncentered second moment matrix*  $\frac{1}{T} R R'$  instead of the (centered) covariance.

# What PCA “Sees” in the Data

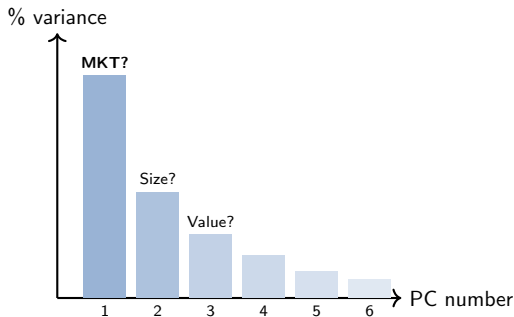
**Intuition:** Each principal component captures one independent dimension of co-movement.

**PC 1:** Typically the “market factor”

- Most stocks move together with the overall market
- Explains the largest fraction of variance (typically 20–30% for individual stocks)

**PC 2:** Often captures size or industry effects

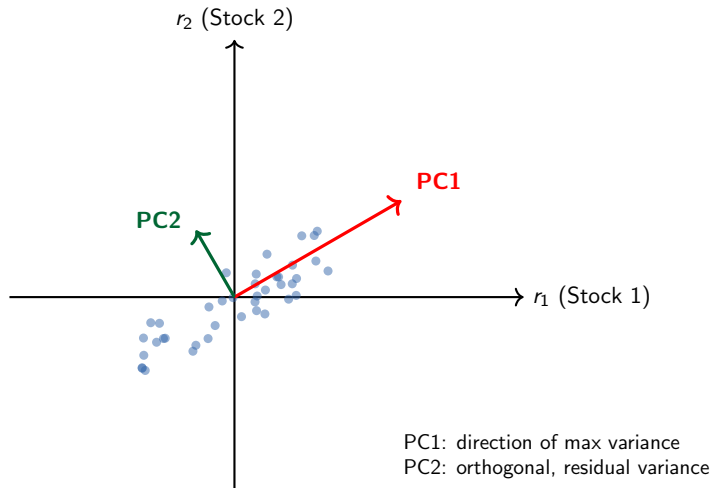
- Orthogonal to PC1 by construction
- Smaller contribution to total variance



Scree plot (schematic)

**Important:** PCA factors are *statistical* objects – they need not correspond to named economic factors. Interpretation requires additional analysis.

# A Numerical Intuition: 2D Example



With  $N = 2$  stocks and  $K = 1$  factor: the first PC captures the main direction of co-movement. The second PC captures residual, orthogonal variation.



# Rotation Indeterminacy – Why It Matters

**Fact:** If  $(\beta, F)$  is a valid factorization, then so is  $(\beta H, H^{-1}F)$  for any invertible  $H$ :

$$r_t = \beta f_t + u_t = (\beta H)(H^{-1}f_t) + u_t$$

The product  $\beta f_t$  is unchanged!

## Consequences:

- PCA factors are **not uniquely identified** – they span the same space as the “true” factors, but may be arbitrary rotations
- We can identify the factor *space* (column space of  $\beta$ ) but not individual factors
- Interpretation requires additional restrictions or rotation criteria

## Subsection 4

### Choosing K

# Choosing the Number of Factors $K$

How many factors do we need?

Several approaches in the literature:

- ① **Information criteria** (Bai & Ng, 2002):

$$IC(K) = \ln \left( \frac{1}{NT} \sum_{i,t} \hat{u}_{i,t}^2 \right) + K \cdot g(N, T)$$

where  $g(N, T)$  is a penalty that grows with  $K$  (analogous to AIC/BIC).

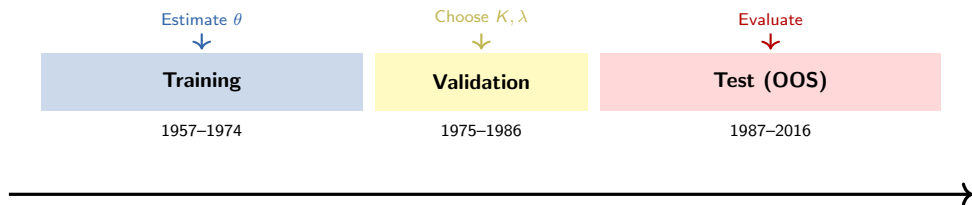
- ② **Eigenvalue ratio test** (Ahn & Horenstein, 2013):

$$ER(K) = \frac{\hat{\lambda}_K}{\hat{\lambda}_{K+1}} \quad (\text{maximize over } K)$$

Large ratio  $\Rightarrow$  big gap between factor  $K$  and factor  $K + 1$ .

- ③ **Cross-validation** (used in the paper): Treat  $K$  as a **tuning parameter** and select via out-of-sample validation performance.

# Cross-Validation for $K$ : The ML Approach



- Try  $K = 1, 2, 3, 4, 5, 6$
- For each  $K$ : estimate the model on training data, evaluate on validation data
- Select the  $K$  that gives the best validation performance
- Report final results on the **test set only** (never used for estimation or tuning)

## Subsection 5

### Limitations of Static Models and Main Takeaways

# Taxonomy of Factor Models (What We Know So Far)

|                       | Observable Factors (FF)                              | Latent Factors (PCA)         |
|-----------------------|------------------------------------------------------|------------------------------|
| <b>Factors</b>        | Pre-specified traded portfolios (MKT, SMB, HML, ...) | Data-driven via SVD          |
| <b>Betas</b>          | Static (constant)                                    | Static (constant)            |
| <b>Cond. info</b>     | None                                                 | None                         |
| <b>Estimation</b>     | OLS time-series regression                           | Eigenvalue decomposition     |
| <b>Interpretation</b> | Clear economic meaning                               | Statistical (needs rotation) |
| <b>Flexibility</b>    | Low                                                  | Moderate                     |
| <b>Missing data</b>   | Handled naturally                                    | Requires EM algorithm        |

**Common weakness:** Both assume  $\beta$  is constant. Neither uses firm characteristics.

# Empirical Evidence: How Do Static Models Perform?

**Table 1 of the paper:** Out-of-sample total  $R^2$  (%) for individual stocks  $r_t$ .

| Model | $K = 1$ | $K = 2$ | $K = 3$ | $K = 4$ | $K = 5$ | $K = 6$ |
|-------|---------|---------|---------|---------|---------|---------|
| FF    | 4.8     | 4.6     | 3.4     | 0.1     | -2.3    | -6.1    |
| PCA   | 7.3     | 3.3     | 5.0     | 5.3     | 4.2     | 3.9     |
| IPCA  | 11.2    | 12.4    | 13.3    | 13.7    | 14.3    | 14.5    |
| CA2   | 10.7    | 11.8    | 12.6    | 13.2    | 13.6    | 13.8    |

## Observations:

- FF models *deteriorate* as  $K$  increases (overfitting with static betas!)
- FF6 has **negative**  $R^2$ : worse than predicting zero for every stock
- PCA is better but plateaus around 5–7%
- Conditional models (grey rows) are dramatically better – we'll see why in Part 3

# Preview: What We Need

## The limitations we've identified:

- 1 **Static betas** miss time-variation in risk exposures  
⇒ *Solution*: Make  $\beta$  a function of observable characteristics  $z_{i,t-1}$
- 2 **Observable factors** may not be the “right” factors  
⇒ *Solution*: Use latent factors estimated from the data
- 3 **Linear models** may not capture the true relationship between characteristics and betas  
⇒ *Solution*: Allow nonlinear mappings (neural networks)
- 4 **Dimensionality reduction** should incorporate both returns and characteristics  
⇒ *Solution*: The *conditional autoencoder* – combining all of the above

## Roadmap:

- Part 2: PCA = linear autoencoder (the mathematical bridge)
- Part 3: Adding conditioning information ⇒ IPCA, then going nonlinear



## Part 1 – Summary

- 1 The **APT** (Ross, 1976) posits that returns follow a linear factor structure  $r_t = \beta f_t + u_t$ , and under no-arbitrage,  $\mathbb{E}[r_i] = \beta_i' \lambda$ .
- 2 **Observable factor models** (Fama–French) use pre-specified traded portfolios. Economically interpretable, but factors are ad hoc and betas are static.
- 3 **PCA** extracts latent factors via SVD:  $\bar{R} = \hat{P} \hat{\Lambda} \hat{Q} + \hat{U}$ . Data-driven, consistent as  $N, T \rightarrow \infty$  (Bai & Ng), but still assumes static loadings.
- 4 The number of factors  $K$  can be chosen via information criteria, eigenvalue ratios, or cross-validation (the paper's approach).
- 5 **Empirically:** Static models (both FF and PCA) perform poorly on individual stocks with infrequent re-estimation and a large cross-section. This motivates *conditional* and *nonlinear* factor models.

**Next (Part 2):** We'll see that PCA is a special case of autoencoders, opening the door to more powerful architectures.

## Part 2

### From PCA to the Linear Autoencoder

## Part 2 – Roadmap

- ➊ Quick recap: what is an autoencoder?
- ➋ The standard autoencoder architecture
- ➌ The encoder–decoder paradigm and the bottleneck
- ➍ Writing the linear autoencoder mathematically
- ➎ linear AE = PCA
- ➏ loadings, factors, and rotation
- ➐ What changes when we add nonlinearity?
- ➑ Deep autoencoders: going beyond PCA
- ➒ The hierarchy:  $\text{PCA} \subset \text{Linear AE} \subset \text{Nonlinear AE} \subset \text{Conditional AE}$

## Subsection 1

### Autoencoder architecture

# What Is an Autoencoder?

**From Lecture 1:** An autoencoder is a neural network trained to *reconstruct its own input*.

## Definition

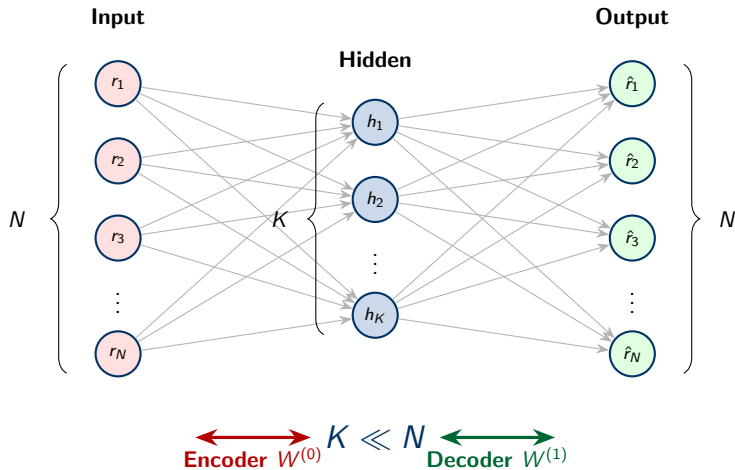
An autoencoder maps input  $x$  through a bottleneck and attempts to produce output  $\hat{x} \approx x$ .

$$x \xrightarrow{\text{encoder}} h \xrightarrow{\text{decoder}} \hat{x} \approx x$$

## Why is this useful?

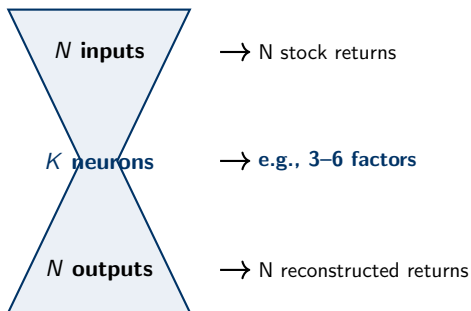
- The bottleneck  $h$  (hidden layer with  $K \ll N$  neurons) forces a *compressed representation*
- If the network can reconstruct  $x$  well from  $h$ , then  $h$  captures the essential information
- This is an **unsupervised** method: no labels needed, only the data itself

# Standard Autoencoder Architecture



# The Bottleneck: Why It Forces Compression

**Key principle:** Information must pass through a narrow channel.



- The network *cannot* simply memorize all  $N$  returns through  $K \ll N$  neurons
- It must learn a **compressed representation** that preserves the most important information
- The “important information” is precisely the **common factor structure**
- Idiosyncratic noise gets discarded (it can't squeeze through the bottleneck)

# The Encoder–Decoder Paradigm

**Encoder** ( $N \rightarrow K$ ):

$$h = g\left(b^{(0)} + W^{(0)}r_t\right)$$

- Takes  $N$ -dimensional return vector
- Compresses to  $K$ -dimensional hidden representation
- $W^{(0)} \in \mathbb{R}^{K \times N}$ : encoder weights
- $g(\cdot)$ : activation function

**Decoder** ( $K \rightarrow N$ ):

$$\hat{r}_t = b^{(1)} + W^{(1)}h$$

- Takes  $K$ -dimensional hidden code
- Reconstructs  $N$ -dimensional return vector
- $W^{(1)} \in \mathbb{R}^{N \times K}$ : decoder weights
- Output layer is **always linear** (no activation)



# The Encoder–Decoder Paradigm

**Full model:**

$$\hat{r}_t = b^{(1)} + W^{(1)} g\left(b^{(0)} + W^{(0)} r_t\right)$$

**If  $g$  is nonlinear** (ReLU): nonlinear autoencoder (more flexible than PCA).

**If  $g$  is identity** ( $g(x) = x$ ): linear autoencoder. **This is what we analyze now.**

## Subsection 2

Linear autoencoder = PCA

# The Linear Autoencoder

Set the activation function to the identity:  $g(x) = x$ .

Linear autoencoder with one hidden layer of  $K$  neurons

$$r_t = b^{(1)} + W^{(1)} \left( b^{(0)} + W^{(0)} r_t \right) + u_t$$

**Parameters:**

$W^{(0)} \in \mathbb{R}^{K \times N}$  encoder weight matrix

$W^{(1)} \in \mathbb{R}^{N \times K}$  decoder weight matrix

$b^{(0)} \in \mathbb{R}^{K \times 1}$  encoder bias

$b^{(1)} \in \mathbb{R}^{N \times 1}$  decoder bias

**Total parameters:**  $K(1 + N) + N(1 + K) = 2NK + N + K$

# Expanding the Model

Expand the parentheses:

$$r_t = \underbrace{b^{(1)} + W^{(1)}b^{(0)}}_{\text{combined bias}} + \underbrace{W^{(1)}W^{(0)}}_{\text{rank-}K \text{ matrix}} r_t + u_t$$

**Observation:** The product  $W^{(1)}W^{(0)}$  is an  $N \times N$  matrix, but it has rank at most  $K$  (because  $W^{(1)}$  is  $N \times K$  and  $W^{(0)}$  is  $K \times N$ ).

**So the autoencoder projects  $r_t$  onto a  $K$ -dimensional subspace, adds a bias, and tries to reconstruct  $r_t$ .**

Compare with the factor model:

$$r_t = \alpha + \underbrace{\beta}_{N \times K} \underbrace{f_t}_{K \times 1} + u_t$$

The autoencoder learns both  $\beta$  (through  $W^{(1)}$ ) and  $f_t$  (through  $W^{(0)}r_t$ ) simultaneously!

# The Optimization Problem

**Training objective:** Minimize reconstruction error over all time periods.

## Optimization Equation

$$\min_{b, W} \sum_{t=1}^T \left\| r_t - \left[ b^{(1)} + W^{(1)} \left( b^{(0)} + W^{(0)} r_t \right) \right] \right\|^2$$

In matrix notation (stacking over time):

$$\min_{b, W} \left\| R - \left[ b^{(1)} \mathbf{1}' + W^{(1)} \left( b^{(0)} \mathbf{1}' + W^{(0)} R \right) \right] \right\|_F^2$$

where  $\| \cdot \|_F$  is the Frobenius norm,  $\mathbf{1}$  is a  $T \times 1$  vector of ones.

**This is a least-squares problem.** For the linear case, we can solve it analytically by taking first-order conditions.

# The Optimization Problem

## Optimal Solution

The optimal solution to the optimization problem is given by:

$$\hat{W}^{(1)} = \hat{P} A \quad (\text{i})$$

$$\hat{W}^{(0)} = \left( \hat{W}^{(1)'} \hat{W}^{(1)} \right)^{-1} \hat{W}^{(1)'} \quad (\text{ii})$$

$$\hat{b}^{(1)} = \bar{r} - \hat{W}^{(1)} \hat{b}^{(0)} - \hat{W}^{(1)} \hat{W}^{(0)} \bar{r} \quad (\text{iii})$$

$$\hat{b}^{(0)} = a \quad (\text{arbitrary constant}) \quad (\text{iv})$$

where  $A$  is any  $K \times K$  nonsingular matrix,  $a$  is an arbitrary scalar,  $\bar{r} = \frac{1}{T} \sum_t r_t$ , and  $\hat{P}$  are the left singular vectors from the SVD of  $\bar{R}$  (i.e., the PCA loadings).

**Bottom line:** The decoder weights  $\hat{W}^{(1)}$  are the PCA loadings (up to rotation  $A$ ).

## Proof – Step 1: Eliminate the Bias Terms

**Take the FOC with respect to  $b^{(1)}$ :**

$$\frac{\partial}{\partial b^{(1)}} \left\| R - b^{(1)} \mathbf{1}' - W^{(1)}(b^{(0)} \mathbf{1}' + W^{(0)} R) \right\|_F^2 = 0$$

Setting the derivative to zero:

$$\left[ R - b^{(1)} \mathbf{1}' - W^{(1)}(b^{(0)} \mathbf{1}' + W^{(0)} R) \right] \mathbf{1} = 0$$

Solving for  $b^{(1)}$ :

$$\hat{b}^{(1)} = \frac{1}{T} \left( R \mathbf{1} - T W^{(1)} b^{(0)} - W^{(1)} W^{(0)} R \mathbf{1} \right) = \bar{r} - W^{(1)} b^{(0)} - W^{(1)} W^{(0)} \bar{r}$$

**Substituting back:**

$$\min_W \left\| \underbrace{R - \bar{r} \mathbf{1}'}_{\bar{R}} - W^{(1)} W^{(0)} \underbrace{(R - \bar{r} \mathbf{1}')}_{\bar{R}} \right\|_F^2 = \min_W \left\| \bar{R} - W^{(1)} W^{(0)} \bar{R} \right\|_F^2$$

The bias terms drop out! The problem reduces to demeaned returns  $\bar{R}$ .

## Proof – Step 1: Key Insight

### What just happened?

The bias terms  $b^{(0)}$  and  $b^{(1)}$  absorb the **mean return**. The remaining optimization involves only **demeaned** returns  $\bar{R}$ .

This is exactly analogous to standard PCA, which is applied to the demeaned (or centered) data!

**Note:**  $b^{(0)}$  is arbitrary – it can take any value  $a$ , and  $b^{(1)}$  adjusts to compensate. Only the product matters:

$$b^{(1)} + W^{(1)}b^{(0)} = \bar{r} - W^{(1)}W^{(0)}\bar{r} = (I - W^{(1)}W^{(0)})\bar{r}$$

This is the part of the mean return that the rank- $K$  projection cannot explain.



## Proof – Step 2: Solve for $W^{(0)}$ Given $W^{(1)}$

**Now minimize over  $W^{(0)}$  with  $W^{(1)}$  fixed:**

$$\frac{\partial}{\partial W^{(0)}} \left\| \bar{R} - W^{(1)} W^{(0)} \bar{R} \right\|_F^2 = 0$$

This gives:

$$W^{(1)'} W^{(1)} W^{(0)} \bar{R} \bar{R}' - W^{(1)'} \bar{R} \bar{R}' = 0$$

Assuming  $W^{(1)'} W^{(1)}$  and  $\bar{R} \bar{R}'$  have full rank, solve:

$$\hat{W}^{(0)} = \left( W^{(1)'} W^{(1)} \right)^{-1} W^{(1)'}$$

**This is the Moore–Penrose pseudoinverse of  $W^{(1)}$ !**

The encoder is the “optimal inverse” of the decoder.

## Proof – Step 2: The Projection Matrix

**Substituting  $\hat{W}^{(0)}$  back:**

$$W^{(1)} \hat{W}^{(0)} = W^{(1)} \left( W^{(1)'} W^{(1)} \right)^{-1} W^{(1)'} =: P_{W^{(1)}}$$

$P_{W^{(1)}}$  is the **orthogonal projection matrix** onto the column space of  $W^{(1)}$ .

So the objective becomes:

$$\min_{W^{(1)}} \left\| \bar{R} - P_{W^{(1)}} \bar{R} \right\|_F^2$$

### Equivalent formulation

**Find the  $K$ -dimensional subspace (column space of  $W^{(1)}$ ) such that projecting  $\bar{R}$  onto it minimizes the reconstruction error.**

This is *exactly* the PCA problem!

## Proof – Step 3: Apply Eckart–Young–Mirsky

**Theorem (Eckart–Young–Mirsky):** The best rank- $K$  approximation of a matrix  $M$  in Frobenius norm is obtained by its truncated SVD:

$$M^{(K)} = \sum_{j=1}^K \sigma_j p_j q_j'$$

**In our case:**  $M = \bar{R}$ , and the best rank- $K$  approximation is  $\hat{P}\hat{\Lambda}\hat{Q}$ .

The projection that achieves this is:

$$P_{W^{(1)}} \bar{R} = \hat{P}\hat{\Lambda}\hat{Q}$$

This is satisfied if and only if  $W^{(1)} = \hat{P} A$  for any invertible  $K \times K$  matrix  $A$ .

### Why the rotation $A$ ?

- $P_{\hat{P}A} = P_{\hat{P}}$  for any invertible  $A$  (same column space)
- The projection depends only on the *subspace*, not on the specific basis
- This is the rotation indeterminacy we saw in Part A

## Proof – Step 3: Putting It Together

Therefore:

$$\hat{W}^{(1)} = \hat{P} A \quad (\text{decoder} = \text{PCA loadings} \times \text{rotation})$$

$$\hat{W}^{(0)} = (\hat{W}^{(1)'} \hat{W}^{(1)})^{-1} \hat{W}^{(1)'} = A^{-1} \hat{P}' \quad (\text{encoder} = \text{rotated pseudoinverse})$$

$$\hat{W}^{(0)} \bar{R} = A^{-1} \hat{P}' \bar{R} = A^{-1} \hat{\Lambda} \hat{Q} \quad (\text{encoded output} = \text{rotated PCA factors})$$

**Reconstruction:**

$$\hat{W}^{(1)} \hat{W}^{(0)} \bar{R} = (\hat{P} A)(A^{-1} \hat{\Lambda} \hat{Q}) = \hat{P} \hat{\Lambda} \hat{Q}$$

The rotation  $A$  cancels out in the final reconstruction!



### Conclusion

The linear autoencoder and PCA produce **identical** reconstructions  $\hat{r}_t$  and span the **same factor space**.

# Summary of the Proof

**Step 1:** FOC for  $b^{(1)} \Rightarrow$  bias absorbs the mean  $\Rightarrow$  work with  $\bar{R}$



**Step 2:** FOC for  $W^{(0)} \Rightarrow \hat{W}^{(0)} = (W^{(1)'} W^{(1)})^{-1} W^{(1)'}$  (pseudoinverse)



**Substituting:**  $\min_{W^{(1)}} \|\bar{R} - P_{W^{(1)}} \bar{R}\|_F^2$  (projection problem)



**Step 3:** Eckart–Young  $\Rightarrow \hat{W}^{(1)} = \hat{P}A$  (PCA loadings up to rotation)

$\Rightarrow$  **Linear AE = PCA**

## Subsection 3

### From Autoencoder Back to Asset Pricing

# Where We Are and Where We're Going

**What we proved (Proposition 1):** The optimal linear autoencoder satisfies

$$\hat{W}^{(1)} = \hat{P}A, \quad \hat{W}^{(0)} = (\hat{W}^{(1)'} \hat{W}^{(1)})^{-1} \hat{W}^{(1)'}$$

**The question now:** We have been working with abstract matrices  $W^{(0)}$ ,  $W^{(1)}$ ,  $\hat{P}$ ,  $\hat{Q}$ ,  $\hat{\Lambda}$ ...

## The risk of losing sight of the economics

At this point, it is easy to forget that we started from a very concrete **financial problem**: explaining stock returns through a small number of latent risk factors.

**Goal of these slides:** Carefully reconnect every autoencoder matrix to the original asset pricing quantities: **returns**, **factors**, and **factor loadings**.

## Recall: The Factor Model We Want to Estimate

The starting point is the **static linear factor model**:

$$r_t = \beta f_t + u_t$$

| Symbol  | Dimension    | Meaning                                               |
|---------|--------------|-------------------------------------------------------|
| $r_t$   | $N \times 1$ | Excess returns of $N$ stocks at time $t$              |
| $f_t$   | $K \times 1$ | Latent factor returns ( $K \ll N$ )                   |
| $\beta$ | $N \times K$ | Factor loadings (how each stock loads on each factor) |
| $u_t$   | $N \times 1$ | Idiosyncratic return (stock-specific noise)           |

**The key economic content:**

- $\beta$  tells us the **risk exposure** of each stock to each factor
- $f_t$  are the **common risk factors** driving all stocks together
- $u_t$  is diversifiable risk – it washes out in large portfolios



# Recall: The Linear Autoencoder We Trained

The one-layer linear autoencoder:

$$\hat{r}_t = b^{(1)} + W^{(1)}(b^{(0)} + W^{(0)}r_t)$$

| Symbol                        | Dimension    | Role in the autoencoder                                  |
|-------------------------------|--------------|----------------------------------------------------------|
| $r_t$                         | $N \times 1$ | Input (raw stock returns)                                |
| $W^{(0)}$                     | $K \times N$ | Encoder weights (compress $N$ returns into $K$ codes)    |
| $b^{(0)}$                     | $K \times 1$ | Encoder bias                                             |
| $h_t := b^{(0)} + W^{(0)}r_t$ | $K \times 1$ | Hidden code (compressed representation)                  |
| $W^{(1)}$                     | $N \times K$ | Decoder weights (reconstruct $N$ returns from $K$ codes) |
| $b^{(1)}$                     | $N \times 1$ | Decoder bias                                             |
| $\hat{r}_t$                   | $N \times 1$ | Output (reconstructed returns)                           |

Now let's map each piece to the factor model.

## First: What Happens to the Bias Terms (Demeaning)

Before identifying autoencoder matrices with factor model quantities, we need to understand a crucial simplification from Step 1 of the proof.

The autoencoder reconstructs:

$$\hat{r}_t = b^{(1)} + W^{(1)}(b^{(0)} + W^{(0)}r_t) = \underbrace{(b^{(1)} + W^{(1)}b^{(0)})}_{\text{combined bias}} + W^{(1)}W^{(0)}r_t$$

Step 1 showed that the **optimal bias** satisfies:

$$b^{(1)} + W^{(1)}b^{(0)} = \bar{r} - W^{(1)}W^{(0)}\bar{r} = (I - W^{(1)}W^{(0)})\bar{r}$$

Substituting back:

$$\hat{r}_t = (I - W^{(1)}W^{(0)})\bar{r} + W^{(1)}W^{(0)}r_t = \bar{r} + W^{(1)}W^{(0)}\underbrace{(r_t - \bar{r})}_{\bar{r}_t}$$

## First: What Happens to the Bias Terms (Demeaning)

### The key consequence

The bias absorbs the mean return  $\bar{r}$ . The weight matrices  $W^{(1)}$  and  $W^{(0)}$  only need to model the **demeaned returns**  $\bar{r}_t = r_t - \bar{r}$ . This is why we work on  $\bar{R}$  (the matrix of demeaned returns), and exactly mirrors PCA, which also operates on centered data.

# Why the Demeaning Matters Economically

The decomposition from the previous slide is:

$$\hat{r}_t = \underbrace{\bar{r}}_{\text{average return}} + \underbrace{W^{(1)}W^{(0)}\bar{r}_t}_{\text{time-varying part}}$$

This has a natural economic interpretation:

| Term                        | Economic meaning                                           |
|-----------------------------|------------------------------------------------------------|
| $\bar{r}$                   | Unconditional expected excess return of each stock         |
| $\bar{r}_t = r_t - \bar{r}$ | Surprise: deviation of returns from their long-run average |
| $W^{(1)}W^{(0)}\bar{r}_t$   | The part of the surprise explained by $K$ common factors   |

**From now on**, we focus on the demeaned returns  $\bar{r}_t$  and study the product  $W^{(1)}W^{(0)}$ . This is where the factor structure lives.

# Why $\hat{W}^{(1)}$ Are the Factor Loadings: The Structural Argument

Working with demeaned returns, the autoencoder reconstruction is:

$$\hat{\bar{r}}_t = \hat{W}^{(1)} \cdot \hat{W}^{(0)} \bar{r}_t$$

Now **define**  $\hat{f}_t := \hat{W}^{(0)} \bar{r}_t$ , which is a  $K \times 1$  vector. Then we can write:

$$\hat{\bar{r}}_t = \hat{W}^{(1)} \hat{f}_t$$

Compare this side-by-side with the factor model:

| Factor model                   | Autoencoder                                 |
|--------------------------------|---------------------------------------------|
| $r_t = \beta f_t + u_t$        | $\hat{\bar{r}}_t = \hat{W}^{(1)} \hat{f}_t$ |
| $\beta$ is $N \times K$        | $\hat{W}^{(1)}$ is $N \times K$             |
| $f_t$ is $K \times 1$          | $\hat{f}_t$ is $K \times 1$                 |
| $\beta$ multiplies the factors | $\hat{W}^{(1)}$ multiplies the factors      |

# Why $\hat{W}^{(1)}$ Are the Factor Loadings: The Structural Argument

## The identification

The two equations have **identical algebraic structure**. In both cases, an  $N \times K$  matrix multiplies a  $K \times 1$  vector to produce reconstructed returns. In the factor model that matrix is  $\beta$ ; in the autoencoder it is  $\hat{W}^{(1)}$ . Therefore  $\hat{W}^{(1)}$  is the autoencoder's estimate of the factor loadings:

$$\hat{W}^{(1)} \longleftrightarrow \hat{\beta}$$

To see this concretely, write the equation **for a single stock  $i$** . The  $i$ -th row of  $\hat{r}_t = \hat{W}^{(1)} \hat{f}_t$  gives:

$$\hat{r}_{i,t} = \hat{W}_{i1}^{(1)} \hat{f}_{1,t} + \hat{W}_{i2}^{(1)} \hat{f}_{2,t} + \cdots + \hat{W}_{iK}^{(1)} \hat{f}_{K,t}$$

The  $i$ -th row of the factor model  $r_t = \beta f_t + u_t$  gives:

$$r_{i,t} = \beta_{i1} f_{1,t} + \beta_{i2} f_{2,t} + \cdots + \beta_{iK} f_{K,t} + u_{i,t}$$

**Each coefficient  $\hat{W}_{ik}^{(1)}$  plays the exact same role as  $\beta_{ik}$ :** it measures the sensitivity of stock  $i$ 's return to factor  $k$ .

# The Encoder Output = Estimated Factors

From the proof (Step 2)

$$\hat{W}^{(0)} = (\hat{W}^{(1)'} \hat{W}^{(1)})^{-1} \hat{W}^{(1)'} = A^{-1} \hat{P}'$$

The **hidden code** (encoder output) at time  $t$ , ignoring bias terms, is:

$$h_t = \hat{W}^{(0)} \bar{r}_t \longleftrightarrow \hat{f}_t \quad (K \times 1)$$

**What is this object?** Each component of  $\hat{f}_t$  is a **weighted average** of all  $N$  stock returns:

$$\hat{f}_{k,t} = \sum_{i=1}^N \hat{W}_{ki}^{(0)} r_{i,t}$$

## Key economic interpretation

Each factor is a **portfolio**: a linear combination of individual stock returns. The encoder has *learned* which portfolio weights best summarize the common variation in returns. These factors are **tradeable** – an investor could, in principle, hold these portfolios.

# The Reconstruction = Factor Model Fit

Putting the two pieces together on demeaned returns:

$$\hat{r}_t = \underbrace{\hat{W}^{(1)}}_{\hat{\beta} \ (N \times K)} \underbrace{\hat{W}^{(0)} \bar{r}_t}_{\hat{f}_t \ (K \times 1)} = \hat{\beta} \hat{f}_t$$

And adding back the mean (which the bias absorbed):

$$\hat{r}_t = \bar{r} + \sum_{k=1}^K \hat{\beta}_k \hat{f}_{k,t}$$

*Each stock's return is approximated as its long-run average plus a linear combination of  $K$  factor returns, weighted by that stock's loadings.*



# The Reconstruction = Factor Model Fit

The **residual** is the idiosyncratic component:

$$\hat{u}_t = r_t - \hat{r}_t = r_t - \hat{\beta} \hat{f}_t$$

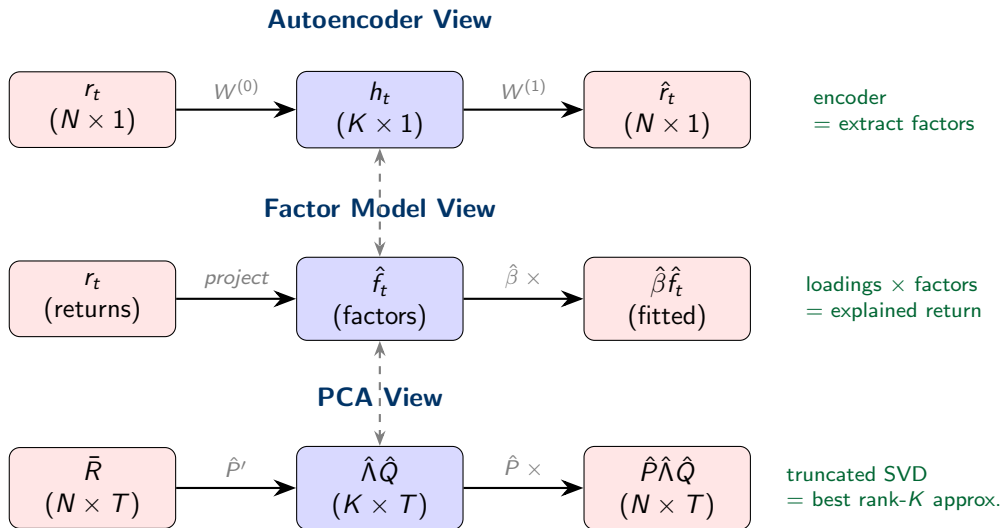
This is the part of the return **not explained** by the  $K$  common factors – the diversifiable, stock-specific risk (see Appendix).

| Autoencoder                                      | Dimension    | Factor Model                            | PCA (SVD of $\bar{R}$ )           |
|--------------------------------------------------|--------------|-----------------------------------------|-----------------------------------|
| $r_t$ (input)                                    | $N \times 1$ | $r_t$ (stock returns)                   | $r_t$                             |
| $\hat{W}^{(1)}$ (decoder)                        | $N \times K$ | $\hat{\beta}$ (loadings)                | $\hat{P} A$                       |
| $\hat{W}^{(0)} \bar{r}_t$ (encoder output)       | $K \times 1$ | $\hat{f}_t$ (factors)                   | $A^{-1} \hat{\Lambda} \hat{Q}_t$  |
| $\hat{W}^{(1)} \hat{W}^{(0)} \bar{r}_t$ (output) | $N \times 1$ | $\hat{\beta} \hat{f}_t$ (fitted return) | $\hat{P} \hat{\Lambda} \hat{Q}_t$ |
| $r_t - \hat{r}_t$ (residual)                     | $N \times 1$ | $\hat{u}_t$ (idiosyncratic)             | $\hat{U}_t$                       |
| $K$ (bottleneck width)                           | scalar       | # of factors                            | # of PCs retained                 |

## The unifying insight

The autoencoder bottleneck, the number of PCA components retained, and the number of latent factors in the asset pricing model are **the same object** viewed from three different perspectives.

# Visual Summary: Three Views of the Same Model



## Subsection 4

### Adding Non-Linearity

# What If We Add Nonlinearity?

The linear autoencoder = PCA. Now consider adding ReLU activation:

## Linear autoencoder:

$$h_t = b^{(0)} + W^{(0)}r_t$$

$$\hat{r}_t = b^{(1)} + W^{(1)}h_t$$

- Equivalent to PCA
- Can only learn **linear** subspaces
- Global optimum (convex problem)

## Nonlinear autoencoder:

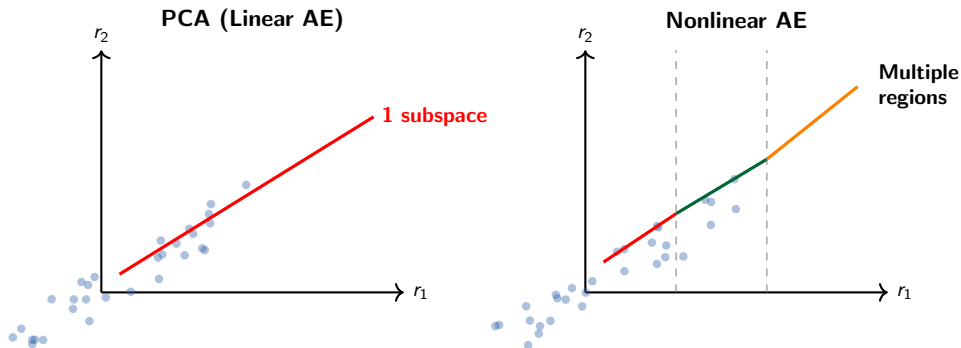
$$h_t = \text{ReLU}\left(b^{(0)} + W^{(0)}r_t\right)$$

$$\hat{r}_t = b^{(1)} + W^{(1)}h_t$$

- **Generalizes** PCA
- Can learn **nonlinear manifolds**
- Multiple local optima (non-convex)

**What ReLU does:**  $g(x) = \max(0, x)$  introduces *piecewise linearity*. Different regions of the input space get different linear projections. This is far more flexible than a single global linear projection.

# Nonlinear AE: Partitioning the Return Space



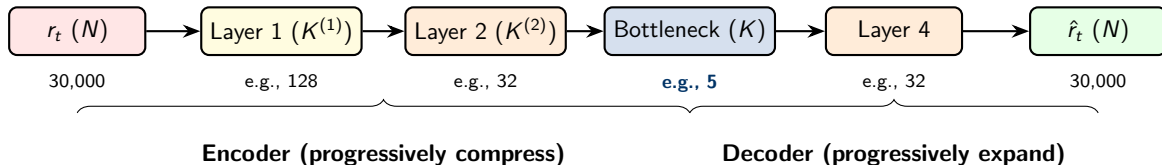
ReLU activation creates **different linear regimes** depending on which neurons are active ( $> 0$ ) or inactive ( $= 0$ ). Each regime has its own effective projection matrix.

# Going Deeper: Multiple Hidden Layers

The general recursive formula:

$$r^{(l)} = g\left(b^{(l-1)} + W^{(l-1)}r^{(l-1)}\right), \quad l = 1, \dots, L$$

**Adding depth** creates a **hierarchy of representations**:

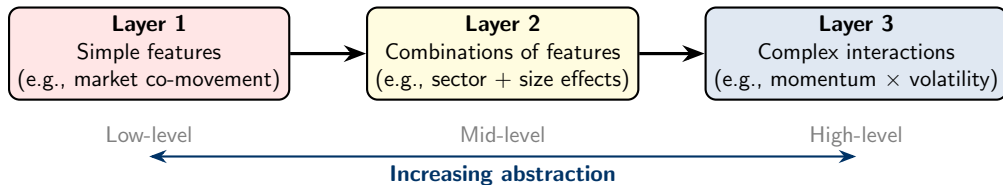


**Deep autoencoders substantially outperform PCA and linear autoencoders for image recognition tasks.**



## Deep AE: What Each Layer Learns

### Intuition from computer vision (transferable to finance):



Each layer builds on the previous one to construct progressively more abstract and useful representations.

**For asset pricing:**

- Early layers: detect basic co-movement patterns
- Later layers: capture complex nonlinear interactions among characteristics
- Bottleneck: compressed factor representation

# Number of Parameters per Architecture

**Capacity** of the model grows with depth and width:

| Model     | Architecture (beta network) | Key feature                   |
|-----------|-----------------------------|-------------------------------|
| Linear AE | No hidden layer             | Equivalent to PCA             |
| CA0       | Single linear layer         | Similar to IPCA               |
| CA1       | 1 hidden layer (32 neurons) | Nonlinear, moderate capacity  |
| CA2       | 2 hidden layers (32, 16)    | Higher capacity, interactions |
| CA3       | 3 hidden layers (32, 16, 8) | Highest capacity              |

**Trade-off:** More parameters  $\Rightarrow$  more flexibility  $\Rightarrow$  but also more risk of overfitting.

This is why **regularization** (L1 penalty, early stopping, ensemble) is critical – as discussed in Lecture 1 and central to the paper's methodology.

# A Note on the Output Layer

**Important design choice:** The output layer of the autoencoder (both standard and conditional) is always **linear** (no activation function).

Final output (Eq. 4 of the paper):

$$G(r, b, W) = b^{(L)} + W^{(L)}r^{(L)}$$

## Why no activation on the output?

- Returns  $r_t$  can be positive or negative, and have no bounded range
- ReLU would clip negative values to zero – destroying half the information
- A linear output allows the reconstruction to match the full range of returns
- The nonlinearity is applied in the **hidden layers** (encoder side), not the output

**This also preserves the economic interpretation:** the output is a linear combination of factor returns weighted by loadings – exactly a factor model.

## Part 2 – Summary

- 1 An **autoencoder** is a neural network that reconstructs its input through a bottleneck. The bottleneck forces a compressed representation.
- 2 With **linear** activation and one hidden layer, the autoencoder is **exactly equivalent to PCA** (Proposition 1): decoder = loadings, encoder output = factors.
- 3 The proof relies on three steps: (i) bias absorbs the mean, (ii) encoder is the pseudoinverse of decoder, (iii) Eckart–Young theorem gives PCA loadings as optimal decoder.
- 4 Adding **nonlinearity** (ReLU) and **depth** (multiple layers) generalizes PCA to capture nonlinear patterns in the data.
- 5 This establishes PCA as a **special case** of the autoencoder framework, and provides a smooth path from linear to nonlinear dimensionality reduction.

**Next (Part C):** How do we incorporate conditioning information ( $z_{i,t-1}$ ) to allow time-varying betas?  $\Rightarrow$  IPCA and the motivation for nonlinear models.

## Part 3

### From Static to Conditional Models

# The Problem: Static Betas Are Inadequate

Standard factor models assume constant loadings:

## Static Factor Model

$$r_{i,t} = \beta' \mathbf{f}_t + u_{i,t} \quad \text{with } \beta \text{ constant across time}$$

But the real world is non-stationary:

- A tech startup in 2001 has very different risk characteristics than in 2010
- A bank's market beta **spikes** during financial crises
- Small-cap stocks may become large-cap over time, changing factor exposures
- Industry composition, leverage, and growth prospects all shift risk profiles

## Result

Fama-French 6-factor model achieves total  $R^2 = -6.1\%$  on individual stocks (Table 1). Static betas with  $\sim 30,000$  stocks, re-estimated only yearly, are badly stale.

# The Problem with Static Betas — A Concrete Example

Consider two firms at the **same point in time**:

*small, distressed, loser*

## Firm A

Market cap: \$50M

B/M ratio: 1.8

Past 12m return: -30%

Leverage: 0.8

*large, growth, winner*

## Firm B

Market cap: \$80B

B/M ratio: 0.2

Past 12m return: +45%

Leverage: 0.1

**Key question:** should Firm A and Firm B have the *same* market beta? The *same* sensitivity to a recession shock?

## The static model assumption

In static factor models (FF, PCA),  $\beta_i$  is **constant** for each firm. Both firms get a single number estimated from 60 months of past returns. This ignores all the information in their current observable attributes.

# Characteristics as Signals of Risk Exposure

Economic theory and empirical evidence suggest that **observable firm attributes predict how a firm responds to systematic shocks**:

| Characteristic                 | Economic rationale                                               | Expected effect on $\beta$                      |
|--------------------------------|------------------------------------------------------------------|-------------------------------------------------|
| <b>Size</b> (market cap)       | Small firms more exposed to liquidity and credit shocks          | Small $\Rightarrow$ higher distress $\beta$     |
| <b>Value</b> (B/M ratio)       | High B/M firms have more assets in place, less flexibility       | High B/M $\Rightarrow$ higher value $\beta$     |
| <b>Momentum</b> (past returns) | Recent winners differ in investor attention, short-sale pressure | High mom $\Rightarrow$ different factor loading |
| <b>Leverage</b>                | More debt amplifies equity sensitivity to macro shocks           | High lev $\Rightarrow$ higher market $\beta$    |
| <b>Profitability</b>           | Profitable firms less exposed to financial distress              | High prof $\Rightarrow$ lower crash $\beta$     |



# Characteristics as Signals of Risk Exposure

## The key insight

Characteristics are **proxies for latent risk exposures**. They contain information about  $\beta$  that is not captured by past returns alone.

# Definition: Firm Characteristics

## Definition

A **firm characteristic**  $z_{i,p,t-1}$  is any observable attribute of firm  $i$ , measured at time  $t - 1$ , that carries information about its future returns or risk exposures.

Characteristics are collected into a vector for each firm and period:

$$z_{i,t-1} = \begin{pmatrix} z_{i,1,t-1} \\ z_{i,2,t-1} \\ \vdots \\ z_{i,P,t-1} \end{pmatrix} \in \mathbb{R}^P$$

They come from **different data sources**, updated at different frequencies:

| Source               | Frequency          | Examples                                 |
|----------------------|--------------------|------------------------------------------|
| Financial statements | Annual / Quarterly | B/M, leverage, profitability, investment |
| Market data          | Monthly            | Size, momentum, volatility, liquidity    |

# Categories of Characteristics

## Price & Trend

mom1m, mom6m,  
mom12m, maxret,  
chmom

## Value

bm, ep, cfp,  
dp, sp

## Profitability

roaq, roic,  
gma, operprof,  
roavol

## Investment

agr, invest,  
egr, chcscho,  
hire

## Risk

beta, betasq,  
retvol, idiovol,  
stdacc

## Liquidity

turn, std\_turn,  
mvel1, dolvol,  
ill, zerotrade

## Financing

acc, chfeps,  
baspread,  
depr, pchgm

## Industry

SIC dummies  
14 indicators

# The Key Idea: Condition Betas on Characteristics

Let betas be a function of observable firm characteristics:

General Conditional Factor Model (Equation 1 of the paper)

$$r_{i,t} = \beta(\mathbf{z}_{i,t-1})' \mathbf{f}_t + u_{i,t}$$

|                                              |                                                                    |
|----------------------------------------------|--------------------------------------------------------------------|
| $\mathbf{z}_{i,t-1} \in \mathbb{R}^P$        | vector of $P$ lagged characteristics of firm $i$ (94 in the paper) |
| $\beta(\mathbf{z}_{i,t-1}) \in \mathbb{R}^K$ | <b>conditional</b> factor loadings — a function of characteristics |
| $\mathbf{f}_t \in \mathbb{R}^K$              | latent factors (still to be estimated)                             |

# The Key Idea: Condition Betas on Characteristics

## What this achieves

As  $\mathbf{z}_{i,t-1}$  changes over time, betas **automatically update** every period.

No re-estimation needed — just plug in new characteristics.

$P$  can be large (94 in the paper) — many sources of conditioning information.

**Open question:** What is the functional form of  $\beta(\cdot)$ ?

# First Solution: Linear Betas (IPCA)

Kelly, Pruitt & Su (2019) propose the simplest functional form: **linear**.

Linear Beta Specification (Equation 2 of the paper)

$$\beta(\mathbf{z}_{i,t-1})' = \mathbf{z}_{i,t-1}' \Gamma \quad \Gamma \in \mathbb{R}^{P \times K} \text{ constant}$$

Each beta is a weighted combination of all 94 characteristics:

$$\beta_{i,k,t-1} = \gamma_k' \mathbf{z}_{i,t-1} = \gamma_1 \cdot \text{size} + \gamma_2 \cdot \text{mom} + \gamma_3 \cdot \text{vol} + \dots$$

**IPCA represents a major advance over static models:**

| Metric ( $K = 6$ , individual stocks) | FF    | PCA  | IPCA |
|---------------------------------------|-------|------|------|
| Total $R^2$ (%)                       | -6.1  | 3.9  | 14.5 |
| Predictive $R^2$ (%)                  | < 0   | < 0  | 0.30 |
| EW Sharpe Ratio                       | -0.21 | 0.15 | 2.25 |

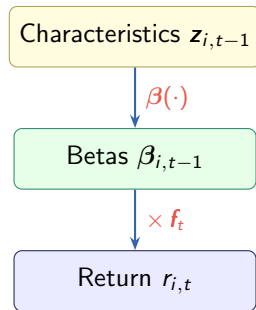
# Why Characteristics $\Rightarrow$ Time-Varying Betas

## Characteristics change over time for each firm:

- A small-cap stock grows  $\rightarrow$  size changes  $\rightarrow$  its  $\beta$  updates
- Leverage increases  $\rightarrow$  amplified exposure  $\rightarrow \beta$  adjusts
- Momentum reverses  $\rightarrow$  new signal  $\rightarrow$  new loadings
- The **same** mapping  $\beta(\cdot)$  applies to all stocks at all times — but each firm's  $\mathbf{z}$  changes

### Key insight

Conditioning information captures both **cross-sectional** and **time-series** variation in risk exposures simultaneously.



### Open question:

What is  $\beta(\cdot)$ ?

# But Is Linearity Enough?

IPCA assumes  $\beta(\mathbf{z}) = \Gamma' \mathbf{z}$  — a linear map. **This rules out:**

## Interactions

$\beta$  depends on size  $\times$  momentum  
*Linear treats each  $z$  independently*

## Thresholds

$\beta$  jumps at distress boundary  
*Linear is always smooth, monotonic*

## Saturation

$\beta$  flattens at extreme values of  $z$   
*Linear extrapolates without bound*

## Curvature

$\beta \propto z^2$  (beta-squared effect)  
*Linear:  $\beta = \gamma z$  misses all curvature*

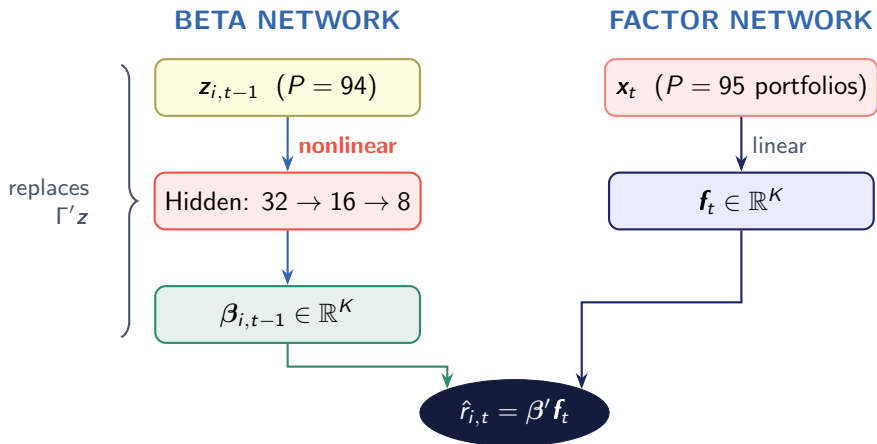
## Theoretical support for nonlinearity

Leading asset pricing models — Campbell–Cochrane (1999), Bansal–Yaron (2004), He–Krishnamurthy (2013), Santos–Veronesi (2004) — all predict that the mapping from state variables to factor betas is **inherently nonlinear**. Linearity is a convenience, not a prediction of theory.



## Next: The Conditional Autoencoder

Replace the linear mapping  $\Gamma$  with a neural network:



# Summary & What's Next

- 1 **Static betas are inadequate:** return distributions are non-stationary; constant  $\beta$  leads to  $R^2 = -6.1\%$  for FF6.
- 2 **Conditioning on characteristics** makes betas time-varying automatically:  $\beta(\mathbf{z}_{i,t-1})$  updates as firm attributes change.
- 3 **IPCA** (linear  $\beta = \Gamma' \mathbf{z}$ ) is a huge improvement — but both theory and data demand nonlinearity.
- 4 **Deep conditional autoencoders** nearly double predictive  $R^2$  and boost VW Sharpe ratios by 60%.
- 5 **Key result:** CA2 matches unrestricted ML prediction while imposing no-arbitrage  $\Rightarrow$  characteristics proxy for **risk exposures**, not anomalies.

## Next Lecture

The Conditional Autoencoder: architecture in detail (CA0  $\rightarrow$  CA3), no-arbitrage restriction, end-to-end training via backpropagation, and full empirical results.

## Appendix

## Subsection 1

Where Did  $u_t$  Go?

## But Wait – Where Did $u_t$ Go?

A natural question arises from the side-by-side comparison:

| Factor model            | Autoencoder                           |
|-------------------------|---------------------------------------|
| $r_t = \beta f_t + u_t$ | $\hat{r}_t = \hat{W}^{(1)} \hat{f}_t$ |

### The question

The factor model has the idiosyncratic term  $u_t$ . The autoencoder does not. Where did it go?

**Answer:** The two equations describe *different things*:

- The factor model describes the **true data-generating process**:  $r_t = \beta f_t + u_t$ . The noise  $u_t$  is always there in reality.
- The autoencoder produces a **reconstruction**  $\hat{r}_t$  – its best attempt to approximate  $\bar{r}_t$ . It *cannot* reproduce  $u_t$  because the bottleneck of  $K \ll N$  neurons does not have enough capacity to pass through  $N$  independent noise terms.

## But Wait – Where Did $u_t$ Go?

The term  $u_t$  has not disappeared. It lives in the **reconstruction error**:

$$\bar{r}_t = \underbrace{\hat{W}^{(1)} \hat{f}_t}_{\text{systematic part}} + \underbrace{(\bar{r}_t - \hat{W}^{(1)} \hat{f}_t)}_{\hat{u}_t \text{ (residual)}}$$

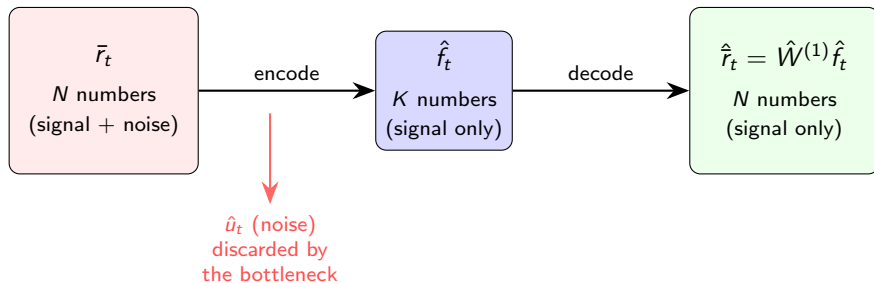
This equation now has *exactly* the same form as  $r_t = \beta f_t + u_t$ .

# The Bottleneck as a Filter for Noise

## Why can't the autoencoder reproduce $u_t$ ?

The idiosyncratic errors  $u_{1,t}, u_{2,t}, \dots, u_{N,t}$  are (approximately) independent across stocks. To perfectly reproduce  $N$  independent numbers, you would need  $N$  degrees of freedom.

But the bottleneck has only  $K$  neurons (e.g.,  $K = 5$ ). So it can transmit at most  $K$  independent pieces of information per time period.



# The Bottleneck as a Filter for Noise

## This is a feature, not a bug

The autoencoder's inability to reproduce noise is precisely what makes it useful. The bottleneck acts as a **filter**: it retains the common factor structure (signal) and discards the idiosyncratic variation (noise). This is the same principle behind PCA, and more broadly, behind all dimension reduction methods.



## Subsection 2

### About the Rotation Indeterminacy

# The Rotation Indeterminacy: Why $A$ Doesn't Matter

Proposition 1 says  $\hat{W}^{(1)} = \hat{P}A$  for *any* invertible  $A$ . Does this mean the solution is not unique?

**Yes, but it doesn't matter.** Here is why.

When we use  $\hat{W}^{(1)} = \hat{P}A$ , the encoder output becomes  $\hat{W}^{(0)}\bar{r}_t = A^{-1}\hat{P}'\bar{r}_t$ . So:

$$\hat{r}_t = \underbrace{(\hat{P}A)}_{\text{loadings}} \underbrace{(A^{-1}\hat{P}'\bar{r}_t)}_{\text{factors}} = \hat{P} \hat{P}' \bar{r}_t$$

The matrix  $A$  **cancels out**. The reconstruction – and therefore the model fit, the residuals, and any  $R^2$  – is identical regardless of  $A$ .

# The Rotation Indeterminacy: Why $A$ Doesn't Matter

## Physical analogy

Think of choosing between Fahrenheit and Celsius. The *temperature* is the same physical quantity; only the *scale* changes. Similarly,  $A$  changes the “coordinate system” within the factor space, but the **factor space itself** – the set of risks that matter – is uniquely determined.

## What is uniquely identified:

- The  $K$ -dimensional factor **subspace** (column space of  $\hat{\beta}$ )
- The fitted returns  $\hat{r}_t = \hat{\beta} \hat{f}_t$
- The residuals  $\hat{u}_t = r_t - \hat{r}_t$